

福 州 大 学

编译系统设计实践

学 生 姓 名： 俞 烨

学 生 学 号： 102301116

专 业 班 级： 计算机类 01 班

指 导 教 师： 何振峰

2026 年 5 月 25 日

摘要

本次实践围绕 Mandrill v2.0 语言编译器的完整实现展开，覆盖词法分析、语法分析、语义检查、代码生成与虚拟机模拟等核心环节。项目采用 Java 作为实现语言，前端从手写词法器与手写递归下降解析器逐步过渡到 ANTLR 4 生成的词法/语法分析器，语义阶段基于符号表与作用域栈完成类型推断、重名检测、参数匹配和控制流合法性检查，后端则面向课程组提供的汇编指令集生成文本汇编并由模拟器执行。

实践分为四个阶段：实验一完成手写词法分析器，能够正确识别关键字、标识符、常量、运算符和界符；实验二实现手写递归下降语法分析器，完成对程序结构、函数定义、语句和表达式的解析；实验三引入 ANTLR 4 工具并实现语义检查器，完成符号表构建、类型检查和作用域管理；实验四则实现编译器后端与虚拟机模拟器，完成代码生成和指令执行。

实验结果表明，编译器能够稳定处理整数、数组、字符串、函数调用、循环与条件分支等常见程序结构，并可对语义错误样例给出明确诊断。通过本次实践，进一步掌握了语言处理系统的分层设计方法，也加深了对作用域管理、动态内存语义和控制流翻译的理解。

关键词：编译原理；Mandrill v2.0；ANTLR；语义检查；代码生成

目录

1	实践目的与实验环境	1
1.1	实践目的	1
1.2	实验环境	1
1.3	详细步骤	1
2	Mandrill 2 语言概述	3
2.1	原理阐述	3
2.2	语言特性	3
2.3	编译流程图	3
2.4	结果分析	4

3 词法分析器	5
3.1 原理阐述	5
3.2 详细步骤	5
3.3 结果分析	6
3.4 问题与解决方案	7
4 语法分析器	8
4.1 原理阐述	8
4.2 详细步骤	8
4.3 结果分析	10
4.4 问题与解决方案	10
5 语义检查器	12
5.1 原理阐述	12
5.2 详细步骤	12
5.3 结果分析	14
5.4 问题与解决方案	14
6 Mandrill 2 虚拟机与代码生成器	15
6.1 原理阐述	15
6.2 详细步骤	15
6.3 编译器与模拟器说明	17
6.4 结果分析	18
6.5 问题与解决方案	18
7 结果分析与问题解决	20
7.1 综合结果分析	20
7.2 主要问题与解决方案汇总	21
8 总结与展望	22
8.1 心得体会	22
8.2 展望	22

A AI 使用情况	23
------------------	-----------

1 实践目的与实验环境

1.1 实践目的

本次课程设计的目标是从零实现一个可运行的 Mandrill v2.0 编译器，并在此基础上完成语义检查与代码生成。具体来说，实践重点包括：理解词法、语法和语义三个层面的语言处理流程；掌握符号表、作用域和函数调用栈的组织方式；学习如何将高级语言结构翻译为面向虚拟机的低层指令；以及通过样例驱动调试提升对编译器整体架构的把控能力。

1.2 实验环境

本项目的主要开发与验证环境如下表所示。

表 1: 实验环境与工具链

项目	说明
操作系统	Linux
开发语言	Java
编译构建工具	Make、javac、java
语法分析工具	ANTLR 4.13.2
目标输出形式	文本汇编指令
运行环境	课程组提供的 Mandrill v2.0 模拟器
编辑与排版	VS Code、LaTeX

项目源码按实验阶段组织为四个模块：实验一完成手写词法分析，实验二完成手写语法分析，实验三引入 ANTLR 并实现语义检查，实验四完成编译器后端与虚拟机模拟器。这样的分阶段组织便于逐步验证每一层的正确性，也便于将错误范围控制在较小的代码块内。

1.3 详细步骤

实践中采用了“先前端、后语义、再后端”的推进顺序。首先根据课程手册梳理语言关键字、表达式和语句结构；随后在词法阶段实现 token 分类与错误处理；再通过语法阶段完成程序、函数、语句块与表达式的结构识别；接着建立符号表和作用域栈完成类型检查；最后面向虚拟机指令集实现代码生成与模拟执行。

在设计过程中，我们始终遵循模块化设计原则，每个阶段独立实现并通过清晰的接口进行数据传递。为保证开发质量，每个实验阶段完成后都会进行独立测试，确保正确后再进入下一阶段。工具链整合也是重要的设计考量，实验三引入 ANTLR 工具以提高开发效率，实验四则复用前端组件减少重复代码。此外，我们坚持错误优先的理念，在词法、语法、语义各阶段及时报告错误，避免错误传播到后续阶段。

2 Mandrill 2 语言概述

2.1 原理阐述

Mandrill v2.0 是一门面向教学场景设计的极简语言，强调“足够表达复杂程序，但尽量减少语言机制”的设计理念。它的核心特征包括：仅使用 32 位整数与数组两类值；数组本质上以指针形式存在；变量类型通过首次使用形式推断；支持局部作用域与全局作用域；提供 `while`、`if`、`break`、`continue`、`return` 等基本控制流；支持函数定义与递归调用；并允许通过 `read/get/put/write` 进行输入输出。

与 C 或 Java 相比，Mandrill 的类型系统更轻量，没有复杂对象模型，也没有显式结构体或类；与 Python 相比，它的语义更接近底层机器模型，尤其强调数组地址、堆内存和控制流跳转。这样的设定使其非常适合作为编译原理课程的训练载体：既保留了语言实现中的关键难点，又不会被过多语法糖干扰。

2.2 语言特性

Mandrill v2.0 中最重要的语义差异体现在以下几个方面：

- 变量类型由首次出现方式决定，例如 `a = 0` 推断为整数，`a[] = read` 推断为数组。
- `local` 与 `global` 明确区分作用域来源，语义检查必须处理同名变量遮蔽问题。
- `read` 对数组和整数的含义不同，数组读取时涉及动态分配与字符串存储。
- 函数参数采用值传递，数组参数传递的是地址；返回值可以是整数或数组地址。
- 字符串和字符字面量使用 UTF-8 源码输入，但编译器内部统一按 UTF-32 处理，便于按定长单元进行寻址。

2.3 编译流程图

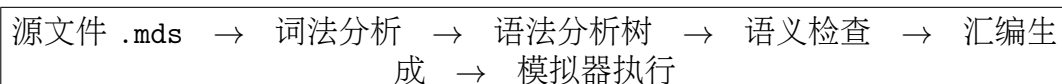


图 1: Mandrill v2.0 编译与执行流程

2.4 结果分析

从语言设计角度看，Mandrill 的约束非常明确，这使得编译器实现可以围绕“整数、数组、函数、控制流”四条主线展开，避免在类型系统或对象模型上分散精力。另一方面，数组与字符串的动态分配、作用域驱动的变量解析、以及 break/continue 的跳转控制，也保证了该语言仍然具备足够的教学挑战。

3 词法分析器

3.1 原理阐述

词法分析器的任务是将字符流切分为 token 流，并跳过注释与空白。实验一采用手写词法分析器，实现了对关键字、标识符、整数常量、字符常量、字符串常量、运算符与界符的识别。由于 Mandrill 允许以 # 开头的单行注释，因此词法分析器还需要在扫描时及时丢弃注释内容。

词法分析器的核心思想是有限状态自动机，通过逐字符扫描并根据当前字符决定状态转移，最终生成相应的 token。关键字采用哈希表存储以实现 O(1) 的查找效率，确保关键字能够优先于标识符被识别。

在设计词法分析器时，我们充分考虑了性能优化的需求，使用哈希表存储关键字以实现 O(1) 的查找效率，避免线性查找带来的性能损耗。同时，错误容错也是重要的设计考量，遇到非法字符时不会立即崩溃，而是记录错误并继续扫描以获取更多诊断信息。位置追踪功能能够记录每个 token 的行号和列号，为后续的错误定位提供精确的位置信息。对于双字符运算符，我们采用前瞻处理技术，需要先读取下一个字符进行判断，例如处理 = 时先检查是否是 ==，如果不是再作为赋值运算符处理。

表 2: Mandrill 2 语言的主要词法单元

类别	示例	说明
关键字	if, while, func, local	具有固定语义，优先于标识符识别
标识符	main, sum, arr1	由字母、数字和下划线组成
常量	123, 'a', "hello"	支持整型、字符与字符串
运算符	+, ==, <=, !=	包括算术、关系与赋值符号
界符	(,), {, }, ;	用于组织语法结构
注释	# ...	从 # 到行尾忽略

3.2 详细步骤

词法阶段的实现策略是基于当前字符进行有限状态扫描：先处理空白与注释，再识别长度较长的多字符运算符，例如 ==、!=、<= 和 >=；接着读取整数和标识符；最后处理字符串与字符常量中的转义序列。为了保证关键字与普通标识符不冲突，扫描完成

后会对候选串进行关键字表查询。

词法分析器的核心扫描算法如下所示：

```
function scanTokens():
    tokens = empty list
    while not end of file:
        read current character
        skip whitespace and comments
        if current is digit:
            scanNumber() and add token
        elif current is letter or '_':
            scanIdentifier() and add token
        elif current is '"':
            scanStringConst() and add token
        elif current is '\':
            scanCharConst() and add token
        elif current is special character:
            scanOperatorOrDelimiter() and add token
    add EOF token
    return tokens
```

对于双字符运算符，采用最长匹配原则：先尝试匹配两字符组合（如 == 、 != ），如果匹配成功则生成对应的 token；否则退回并匹配单字符运算符。字符和字符串常量则需要专门处理转义序列。

3.3 结果分析

实验一阶段，词法器已经能够稳定输出课程组给定样例的 token 序列，并为后续语法分析提供统一输入。对加法表达式、求和函数和字符串处理等样例，词法结果能够正确区分运算符、函数名、字符串与注释内容，说明词法分类是正确的。通过自动化测试脚本可以批量验证所有测试用例，确保词法分析器的正确性。

3.4 问题与解决方案

实现过程中最容易出错的部分有两个：一是双字符运算符需要优先于单字符运算符识别，二是字符串与字符常量中的转义序列必须完整保留。

针对双字符运算符的处理，采用了前瞻（lookahead）技术：

```
function scanOperator():
    if current == '=':
        next = peek next character
        if next == '=':
            consume next, return EQ token
        else:
            return ASSIGN token
    elif current == '!=':
        next = peek next character
        if next == '=':
            consume next, return NEQ token
    # ... 其他运算符处理
```

对于转义序列，在扫描字符串和字符常量时，遇到反斜杠后会特殊处理后续字符，正确转换 `\n`、`\t`、`\"` 等转义字符。

4 语法分析器

4.1 原理阐述

语法分析器负责判断 token 流是否符合 Mandrill 的文法结构，并构建抽象语法树或解析树。实验二实现了手写递归下降语法分析器，实验三则切换为 ANTLR 4 生成的词法/语法分析器，并通过带标签的文法替代了大量手工递归下降代码。最终文法包含程序、函数定义、语句块、赋值语句、循环语句、条件语句、跳转语句和表达式等核心结构。

递归下降分析器的核心思想是将每个文法规则对应到一个递归函数，通过函数调用实现语法规则的推导。这种方法直观易懂，便于手动实现和调试。

在设计语法分析器时，我们面临多个重要的设计权衡。实验二采用手写递归下降分析器，目的是深入理解语法分析的原理和实现细节；实验三则引入 ANTLR 工具生成分析器，以提高开发效率和降低维护成本。表达式优先级通过分层函数调用来实现，不同优先级的运算符由不同层级的函数处理，确保表达式按照正确的顺序进行解析。错误恢复方面，我们采用了简单的 panic 模式，遇到错误时跳过到下一个语句边界，以尽可能多地报告错误信息。对于数组赋值等特殊语法结构，需要具备前瞻能力，预读多个 token 进行判断，以正确识别语法结构。

表 3: 语法分析中的核心结构

结构	作用
program	描述全局作用域中的函数定义与语句序列
functionDef	规定函数名、参数列表和函数体
stmtBlock	描述由花括号包围的语句块
expression	处理字面量、变量、函数调用和二元运算
loopStatement	对应 while 循环结构
conditionStatement	对应 if-else 结构

4.2 详细步骤

实验二的手写语法分析器采用了递归下降的方法，每个语法规则对应一个递归函数。表达式处理采用分层设计，通过不同层级的函数处理不同优先级的运算符：

```
function parse():
    while not EOF:
```

```
        if current token is FUNC:
            parseFunctionDef()
        else:
            parseStatement()

function parseExpression():
    return parseEquality()

function parseEquality():
    left = parseComparison()
    while current token is == or !=:
        op = consume current token
        right = parseComparison()
        return BinaryExpr(op, left, right)
    return left

function parseComparison():
    left = parseAdditive()
    while current token is <, <=, >, >=:
        op = consume current token
        right = parseAdditive()
        return BinaryExpr(op, left, right)
    return left

function parseAdditive():
    left = parseMultiplicative()
    while current token is + or -:
        op = consume current token
        right = parseMultiplicative()
        return BinaryExpr(op, left, right)
    return left
```

```
function parseMultiplicative():
    left = parsePrimary()
    while current token is *, /, %:
        op = consume current token
        right = parsePrimary()
        return BinaryExpr(op, left, right)
    return left
```

实验三引入的 ANTLR 版本采用了词法规则与语法规则分离的设计。表达式部分通过带标签的左递归写法统一描述优先级，从而让乘除模、加减、比较与相等比较按照语言期望的顺序组织。语句部分则将赋值、循环、条件和跳转分别拆分，以便语义检查和代码生成时按节点类型进行分派处理。

在实现上，解析器的关键是确保语法树的结构能够稳定反映程序意图。例如 `a[b]` 在语法层面既可能作为读取数组元素的右值，也可能作为左值出现在赋值语句中；数组返回函数则需要与普通函数的返回值类型一并解析。访问者模式使这些节点可以按类型单独处理，降低了后续阶段的耦合度。

4.3 结果分析

语法分析阶段的价值主要体现在两个方面：一是将大量嵌套表达式和语句块转换为结构化树，便于后续遍历；二是把语法错误尽早暴露在输入阶段，避免无效程序进入语义和后端。对课程组给出的合法样例，如递归求最大公约数、斐波那契数列和字符串处理程序，解析树结构均与预期一致。通过自动化测试脚本可以验证语法分析结果，确保测试用例能正确解析或报告语法错误。

4.4 问题与解决方案

语法阶段最常见的问题是二义性与优先级错误。比如如果不使用分层表达式规则，`a + b * c` 容易被错误地解析成 `(a + b) * c`。解决方法是在文法中明确表达式层级，在手写递归下降分析器中，通过将表达式处理分解为多个层级的函数，从语法层面保证了运算优先级。

对于数组赋值语句 `a[] = expr` 的特殊语法，需要在解析时提前预读多个 token 来识别这种结构：

```
function parseAssignment():
    if lookahead(IDENTIFIER, LBRACKET, RBRACKET, ASSIGN):
        name = consume IDENTIFIER
        consume LBRACKET, RBRACKET, ASSIGN
        expr = parseExpression()
        return ArrayAssign(name, expr)
    else:
        lvalue = parseLValue()
        consume ASSIGN
        expr = parseExpression()
        return Assignment(lvalue, expr)
```

这种前瞻技术确保了数组赋值语句能够被正确识别，避免与普通数组访问混淆。

5 语义检查器

5.1 原理阐述

语义检查器负责在语法正确的基础上进一步判断程序是否合法。实验三的实现基于符号表、作用域栈和访问者模式完成，核心任务包括：变量和函数是否已声明、局部与全局同名冲突、数组与整数的类型一致性、函数参数个数和参数类型匹配、循环外跳转语句的非法使用、以及函数返回值与返回语句的一致性。

语义检查采用访问者模式遍历语法树，在遍历过程中维护符号表和作用域栈，对每个节点进行类型检查和语义验证。符号表负责记录变量和函数的声明信息，包括名称、类型和作用域层级。

在设计语义检查器时，我们采取了多项重要的设计决策。为避免前向引用问题，我们采用两阶段检查策略：先收集所有符号定义，再进行类型检查。作用域管理采用栈式结构，能够有效管理嵌套作用域，支持局部变量遮蔽全局变量的语义。类型推断机制根据变量的首次赋值自动推断其类型，无需显式声明，简化了语言的使用。上下文追踪功能记录当前函数和循环的嵌套层级，用于验证跳转语句的合法性，确保 `break`、`continue` 和 `return` 语句出现在正确的上下文中。

表 4: 典型语义错误与处理方式

错误类型	触发示例	处理方式
未定义变量	<code>x = y + 1;</code> 中 <code>y</code> 未声明	报告未定义标识符
数组/整数混用	对整数变量使用 <code>[]</code> 访问	报告类型不匹配
参数个数错误	调用时实参与形参数目不同	报告参数数量不一致
循环外跳转	在函数体外使用 <code>break</code>	报告跳转语句非法
返回值错误	整数函数返回数组或字符串	报告返回类型不一致

5.2 详细步骤

语义分析采用两层结构：首先在符号收集阶段把函数、全局变量、局部变量和参数登记到符号表中；随后在检查阶段按语法树自顶向下遍历，维护当前所处的函数层数与循环层数，并依据节点类型实施规则判断。

符号收集阶段的核心算法如下:

```
function collectSymbols(node, scope):  
    if node is FunctionDef:  
        func = new FunctionSymbol(name, returnType)  
        scope.define(func)  
        enterFunctionScope()  
        for param in parameters:  
            paramSym = new VariableSymbol(name, type)  
            currentScope.define(paramSym)  
        collectSymbols(body, currentScope)  
        exitFunctionScope()  
    elif node is DeclarationStmt:  
        var = new VariableSymbol(name, type)  
        scope.define(var)  
    elif node is AssignmentStmt:  
        if target is new variable:  
            type = inferType(rvalue)  
            var = new VariableSymbol(name, type)  
            scope.define(var)  
    for child in children:  
        collectSymbols(child, currentScope)
```

符号查找采用作用域链搜索机制:

```
function lookup(name):  
    current = currentScope  
    while current is not null:  
        if current contains name:  
            return current.get(name)  
        current = current.parent  
    if globalScope contains name:  
        return globalScope.get(name)  
    return null
```

在检查过程中，通过两个计数器维护当前上下文，用于验证跳转语句是否出现在合法的上下文内。变量访问时通过符号表逐级查找确定变量的作用域和类型，函数调用时对比实参与形参的数量和类型是否匹配。

5.3 结果分析

实验三阶段提供的语义错误样例涵盖了数组赋值、函数参数、返回值、循环和全局语句等多个维度。对这些样例逐项检查后，语义检查器能够在错误源头给出较明确的报错信息，说明符号表设计与作用域管理是有效的。尤其是在函数参数和数组访问的类型判断上，前端已经能把大量潜在后端崩溃提前拦截掉。

通过自动化测试脚本可以验证语义检查功能，确保所有合法样例通过检查，所有错误样例被正确识别并报告相应的语义错误，位置信息精确到行和列。

5.4 问题与解决方案

语义阶段最棘手的问题是同一名字在不同作用域中的真实含义。例如局部变量与全局变量同名时，后端必须按当前作用域优先解析，否则会把局部定义错误地映射到全局地址。对此，项目中通过作用域栈查找和全局符号回退相结合的方式解决，实现了符号的栈式管理和逐级查找。

另一个问题是数组访问结果在语义上属于整数，但数组变量本身又是指针，这要求检查器在返回类型和取值类型之间做出区分。解决方案是引入三种类型状态：整数类型、数组指针类型和未知类型。在检查变量访问时，根据是否有数组下标访问返回不同的类型信息——数组元素访问返回整数类型，而数组变量本身返回指针类型。

6 Mandrill 2 虚拟机与代码生成器

6.1 原理阐述

后端的目标是把语法树和语义信息翻译为课程组提供的虚拟机汇编。实验四的编译器采用直接遍历语法树生成文本汇编的方式，运行时再交由模拟器解释执行。该方案的优点是实现路径清晰：一旦语义检查通过，代码生成主要关注指令选型、栈帧管理、跳转标签和动态内存申请即可。

代码生成器采用访问者模式遍历语法树，根据节点类型生成相应的虚拟机指令序列。虚拟机模拟器则负责解析汇编指令并执行，包含指令解析器、内存管理器和指令执行引擎。

Mandrill 虚拟机使用一组简洁但足够表达控制流和内存操作的指令，例如全局数据访问指令用于读写全局变量，局部变量访问指令用于读写函数参数和局部变量，算术与比较运算指令完成计算操作，函数调用与返回指令实现过程调用，内存分配指令负责堆内存申请，输入输出指令实现数据的输入输出。

在设计后端时，我们做出了多项关键决策。指令选择上采用栈式虚拟机模型，这种模型能够简化表达式求值和函数调用的实现，使得代码生成更加直观。标签管理使用字符串标签和延迟解析技术，避免手动计算指令地址，提高了代码的可读性和可维护性。内存布局方面，全局变量和局部变量分离管理，函数参数通过栈传递，确保了内存访问的安全性和效率。字符串处理统一采用 UTF-32 编码，每个字符占 4 字节，便于进行随机访问和数组操作。

表 5: 常见语法结构到虚拟机指令的映射

语言结构	生成策略
整数字面量	直接生成 <code>dstore</code> 载入常量
变量读写	根据作用域选择全局或局部访问指令
数组元素访问	计算地址后使用数组读写指令
函数调用	实参逆序入栈，设置帧大小后执行跳转
<code>while</code>	条件、循环体、退出点三标签控制
<code>if-else</code>	<code>then/else/end</code> 三标签控制

6.2 详细步骤

代码生成过程可以概括为以下几个步骤：

1. 先跳转到主入口，避免函数定义体被当作全局语句立即执行。
2. 逐个生成函数体代码，并在函数入口处记录标签。
3. 对语句块维护作用域栈，局部变量与参数使用局部写入指令，全局变量使用全局写入指令。
4. 对条件语句和循环生成条件标签、分支标签和结束标签，利用标签解析实现跳转。
5. 对数组访问生成地址计算序列，即基址加索引乘 4 的偏移量，再使用数组读写指令。
6. 对字符串字面量按 UTF-32 进行展开，并在堆上分配连续空间后逐字符写入。

代码生成器通过循环上下文栈记录嵌套循环的信息，用于正确生成跳转语句的目标；通过作用域栈维护当前作用域，区分局部变量与全局变量的访问方式。字符串字面量专门处理，先计算所需字节数，然后分配堆空间，最后逐个字符写入。

循环语句的代码生成算法如下：

```
function compileWhile(condition, body):  
    condLabel = newLabel("while_cond")  
    bodyLabel = newLabel("while_body")  
    exitLabel = newLabel("while_exit")  
  
    pushLoopContext(condLabel, exitLabel)  
  
    emitLabel(condLabel)  
    emit("dstore", exitLabel)  
    emit("dstore", bodyLabel)  
    compileExpression(condition)  
    emit("eval", CONDITION)  
  
    emitLabel(bodyLabel)  
    compileStatement(body)  
    emit("jump", condLabel)
```

```
emitLabel(exitLabel)
popLoopContext()
```

函数调用的代码生成算法:

```
function compileCall(name, arguments):
    func = symbolTable.lookup(name)
    for arg in reversed(arguments):
        compileExpression(arg)
    emit("dstore", func.frameSize)
    emit("jal", func.label)
    return func.returnType
```

6.3 编译器与模拟器说明

编译器读取 `.mds` 源文件并输出 `.asm` 汇编, 模拟器读取汇编并按指令逐条执行, 最终通过标准输入输出完成程序验证。

图 2: 编译器与模拟器分工

编译器的前端部分复用了实验三的词法分析、语法分析和语义检查, 后端则新增了代码生成模块。模拟器部分包含指令解析器、内存管理器和指令执行引擎, 每条指令对应一个独立的实现类, 通过统一的接口进行调度。

虚拟机模拟器的执行流程如下:

```
function execute(instructions):
    pc = 0
    while pc < length(instructions):
        inst = instructions[pc]
        pc++
        executeInstruction(inst)
```

6.4 结果分析

后端能够稳定通过课程组提供的代表性合法样例，包括加法、表达式求值、递归求最大公约数、斐波那契数列以及字符串相关样例。对这些程序的执行结果进行比对后，汇编输出与答案文件一致，说明函数调用、局部变量、递归返回、数组读写和字符串输出等关键路径已经打通。对于语义错误样例，前端会在生成汇编之前停止，从而避免错误程序进入模拟器执行。

通过自动化测试脚本可以完成从编译到运行的全流程验证：先编译 Mandrill 源程序生成汇编代码，再运行模拟器执行汇编，并将输出与预期答案进行比对，确保所有测试用例都能通过。

6.5 问题与解决方案

后端最重要的难点集中在三个方面。第一是跳转目标的管理，如果不提前分配标签，跳转语句与条件分支很容易错位；因此实现中采用标签栈记录循环上下文，保存循环的开始和结束标签地址。

第二是数组与字符串的动态内存分配，整数读取和数组读取的语义并不一样，最终通过在输入阶段区分目标类型并结合内存分配指令完成。对于整数变量，调用整数输入指令；对于数组变量，调用字符串输入指令，该指令会自动分配堆内存。

第三是字符与字符串的编码处理，课程要求按 UTF-32 按字节寻址，因此字符串字面量在生成时必须统一转换并按 4 字节间隔写入。字符串生成算法如下：

```
function emitStringLiteral(str):
    codePoints = getCodePoints(str)
    bytes = (length(codePoints) + 1) * 4
    emit("dstore", bytes)
    emit("malloc", 0)
    temp = allocateTemp()
    emit("dwrite", temp)

    for i from 0 to length(codePoints)-1:
        emit("dstore", codePoints[i])
        emit("dload", temp)
```

```
    emit("dstore", i * 4)
    emit("eval", ADD)
    emit("dawrite", 0)

emit("dload", temp)
```

该算法先计算字符串所需的字节数（每个码点 4 字节，加上结尾零），然后分配堆空间，最后逐个将字符写入堆内存。

7 结果分析与问题解决

7.1 综合结果分析

从整体上看，本次实践按照“词法-语法-语义-后端”逐层推进的方式完成了 Mandrill v2.0 编译器的实现。前端把复杂输入转换为结构化树，语义层把类型与作用域问题提前拦截，后端把剩余的合法程序映射为可执行汇编，这种分层设计既便于单元验证，也降低了联调难度。

结合课程组提供的样例目录可以看出，编译器覆盖了普通算术与表达式计算、递归函数调用、数组与字符串处理、语义错误检测等典型场景，而且对每类场景都能给出符合预期的结果，说明实现已经达到课程设计要求。四个实验阶段的实现成果如下：

表 6: 各实验阶段实现成果

实验阶段	核心功能	验证结果
实验一	手写词法分析器	正确识别所有词法单元，通过所有测试样例
实验二	手写语法分析器	正确解析程序结构，正确处理表达式优先级
实验三	语义检查器	正确检测类型错误、作用域错误、参数错误
实验四	代码生成器与模拟器	正确生成汇编代码，正确执行并输出预期结果

7.2 主要问题与解决方案汇总

表 7: 实现过程中的典型问题与解决方案

问题类别	解决方案
关键字与标识符冲突	采用优先级扫描和关键字表查询, 确保关键字优先被识别
表达式优先级错误	使用分层文法和递归下降函数, 从语法层面保证运算优先级
作用域解析混乱	采用符号表与作用域栈的组合, 实现逐级查找
函数调用与返回管理复杂	计算函数帧大小, 参数逆序入栈, 统一返回指令
字符串和数组的内存布局不一致	采用统一的 UTF-32 编码和堆分配逻辑
跳转目标管理困难	使用标签栈记录循环上下文, 正确处理跳转语句

8 总结与展望

8.1 心得体会

通过本次课程设计，我对编译器的分层结构有了更具体的认识。以前对“词法、语法、语义、代码生成”这些概念的理解偏抽象，而在真正实现之后，能够明显感受到每一层都有各自独立但又彼此依赖的职责。尤其是作用域、符号表和类型检查的设计，使我对语言实现中的“名字解析”与“值计算”之间的区别有了更清晰的理解。

在实现过程中，我深刻体会到了模块化设计的重要性。每个阶段都有明确的输入输出接口，便于独立开发和测试。同时，通过逐步验证的方式，可以将错误范围控制在较小的代码块内，提高调试效率。

8.2 展望

如果继续扩展该项目，后续可以从以下几个方向完善：其一，增强错误恢复能力，使编译器能够在遇到局部语法错误后继续分析更多内容；其二，引入更丰富的静态分析，例如数组越界检测或未初始化使用检查；其三，改进后端生成策略，加入简单优化以减少无效指令；其四，继续丰富测试集，使回归验证更加系统化。总体而言，Mandrill v2.0 编译器已经为进一步研究更复杂的语言特性打下了良好基础。

参考文献

- [1] Alfred V. Aho, Monica S. Lam, Ravi Sethi, Jeffrey D. Ullman. *Compilers: Principles, Techniques, and Tools*. Pearson, 2nd Edition.
- [2] Terence Parr. *The Definitive ANTLR 4 Reference*. Pragmatic Bookshelf.
- [3] 编译原理课程组. *Mandrill v2.0 项目及语法概览*.
- [4] Oracle. *Java Platform, Standard Edition Documentation*.

A AI 使用情况

在四个实验的代码生成与修改过程中，AI 工具主要承担定位、补全和验证的辅助角色。整体上，使用方式都比较克制：先说明当前要修正的具体模块，再让工具给出局部修改建议，最后结合课程样例和编译结果确认是否可用。

实验一阶段主要围绕词法分析器展开，重点处理词法单元分类、关键字优先级、双字符运算符和注释跳过逻辑；实验二阶段主要用于检查递归下降解析器的语句分派、表达式优先级和函数体组织方式；实验三阶段则更多用在语法分析工具、符号表和作用域栈上，常见问题包括局部变量遮蔽全局变量、数组下标类型不符、函数参数数量不一致等；实验四阶段主要协助后端代码生成与模拟器联调，例如函数调用前的压栈顺序、循环跳转标签维护，以及输入语句对应的动态分配处理。总体来看，AI 工具并没有替代实现工作，而是帮助我更快地发现问题、缩小修改范围并验证结果，最终方案仍然以课程要求、源码结构和样例输出为准。